

Trabajo: Proyecciones 3D

Cuatro vistas de un mismo cubo en OpenGL

Sergio Andrés Majé Franco

Informática Gráfica y Visualización
Universidad Internacional de La Rioja (UNIR)
28 de mayo de 2026

Índice

1. Introducción	2
2. Alcance de la entrega	2
3. Descripción general de la solución	2
4. Implementación de las proyecciones	3
4.1. Proyección ortogonal	3
4.2. Proyección gabinete	3
4.3. Perspectiva simétrica	3
4.4. Perspectiva oblicua	4
5. Evidencia de ejecución	5
6. Visualización complementaria: hiperboloide de una hoja	5
7. Verificación de requisitos	7
8. Funcionamiento del programa	7
8.1. ¿Funciona bien el programa?	7
8.2. ¿Qué fallos existen?	8
9. Conclusiones	8
10. Archivos adjuntos	9

1. Introducción

El presente documento describe la solución desarrollada para el trabajo *Proyecciones 3D*. El informe se concentra en explicar la implementación realizada en OpenGL, presentar la captura del programa en ejecución y responder las preguntas solicitadas en la entrega. La redacción se mantiene enfocada en el resultado construido, sin extenderse hacia una investigación teórica externa al alcance de la actividad.

2. Alcance de la entrega

La actividad solicita una ventana OpenGL con cuatro vistas de un mismo cubo: proyección ortogonal, proyección gabinete, proyección perspectiva simétrica y proyección perspectiva oblicua. Además, el enunciado establece que la proyección debe realizarse sin mover el ojo desde la posición por defecto $(0, 0, 0)$, por lo que no se debe utilizar `gluLookAt()` ni una función equivalente de cámara.

La solución se entrega mediante un archivo `main.c`, en el que se implementa la ventana, la división en cuadrantes, las matrices de proyección y el dibujo de la escena. Este informe funciona como documento explicativo de la implementación y como evidencia visual del resultado obtenido.

3. Descripción general de la solución

El programa utiliza OpenGL clásico sobre X11/GLX. La ventana principal se divide en cuatro cuadrantes y cada cuadrante renderiza el mismo cubo con una configuración de proyección diferente. La geometría de la escena se mantiene común: un cubo con caras coloreadas, aristas resaltadas y ejes de referencia. Esta decisión permite que la comparación se centre en el efecto de las proyecciones y no en variaciones del objeto.

La estructura del código separa tres responsabilidades principales. Primero, se crea la ventana y el contexto gráfico. Luego, se definen funciones para construir matrices ortográficas, matrices de *frustum* y una matriz de gabinete basada en cizalla. Finalmente, cada vista configura su propio viewport, carga su matriz de proyección y dibuja la escena común.

```
1 configure_viewports(app, viewports);
2 glEnable(GL_SCISSOR_TEST);
3
4 drawOrthogonalView(&viewports[0]);
5 drawCabinetView(&viewports[1]);
6 drawSymmetricPerspectiveView(&viewports[2]);
7 drawObliquePerspectiveView(&viewports[3]);
8
9 draw_overlay(app, viewports);
```

Listing 1: Renderizado independiente de las cuatro vistas.

El uso de funciones separadas facilita la revisión técnica, ya que cada proyección puede verificarse de manera aislada. También evita que la matriz de una vista afecte a las demás, debido a que cada cuadrante reinicia explícitamente las matrices de proyección y de modelo antes de dibujar.

4. Implementación de las proyecciones

4.1. Proyección ortogonal

La vista ortogonal se implementa mediante una matriz ortográfica. En esta proyección, los rayos proyectantes son paralelos y, por tanto, las aristas paralelas del cubo se conservan paralelas en la imagen final. Para que la figura sea más legible, el cubo se rota con una pose similar a una vista axonométrica, pero la proyección sigue siendo paralela y no genera puntos de fuga.

La matriz usada corresponde a la forma general de una proyección ortográfica:

$$O = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.2. Proyección gabinete

La proyección gabinete se desarrolló como una proyección oblicua paralela. La intención es conservar la cara frontal del cubo en verdadera magnitud o con una deformación mínima, mientras la profundidad se representa mediante una dirección diagonal reducida. Para lograrlo, se combina una proyección ortográfica con una matriz de cizalla sobre la profundidad.

La transformación conceptual aplicada es:

$$x' = x + kz \cos(\alpha), \quad y' = y + kz \sin(\alpha)$$

En la implementación se utiliza $k = 0,5$ y $\alpha = 45^\circ$. El valor $k = 0,5$ corresponde a la reducción típica de una proyección gabinete, lo que evita que la profundidad se vea exagerada. Como el cubo se ubica delante del observador sobre valores negativos de z , el signo de la cizalla se ajusta para que la profundidad retroceda visualmente en diagonal.

4.3. Perspectiva simétrica

La perspectiva simétrica se implementa con una matriz de *frustum* centrada. En esta vista se cumple que los límites laterales y verticales del volumen de visión son simétricos, es decir, $l = -r$ y $b = -t$. De esta manera, el volumen de proyección queda alineado con el eje de visión.

El cubo no se visualiza mediante movimiento de cámara. En cambio, se traslada como objeto hacia el eje $-Z$ para quedar dentro del volumen visible. Esta decisión respeta la restricción de mantener el ojo en $(0, 0, 0)$ y permite que la reducción por profundidad sea consecuencia de la matriz de perspectiva.

4.4. Perspectiva oblicua

La perspectiva oblicua se resolvió como una perspectiva *off-axis*. A diferencia de la perspectiva simétrica, esta vista usa un *frustum* asimétrico, con límites que no cumplen $l = -r$ o $b = -t$. El resultado es una perspectiva desplazada respecto al eje óptico, sin necesidad de mover el ojo ni de usar `gluLookAt()`.

La diferencia principal frente a la perspectiva simétrica está en la matriz de proyección, no en crear otro cubo ni en cambiar arbitrariamente la cámara. Por eso, el código conserva una colocación similar del objeto en ambas vistas de perspectiva y hace que el contraste dependa del volumen de visión.

```
1 /* Perspectiva simetrica: frustum centrado */
2 build_frustum_matrix(-right, right, -top, top, 3.0, 24.0, projection);
3
4 /* Perspectiva oblicua: frustum asimetrico u off-axis */
5 build_frustum_matrix(-1.65 * right, 0.35 * right,
6                     -1.05 * top, 0.95 * top,
7                     3.0, 24.0, projection);
```

Listing 2: Diferencia entre frustum simétrico y frustum asimétrico.

5. Evidencia de ejecución

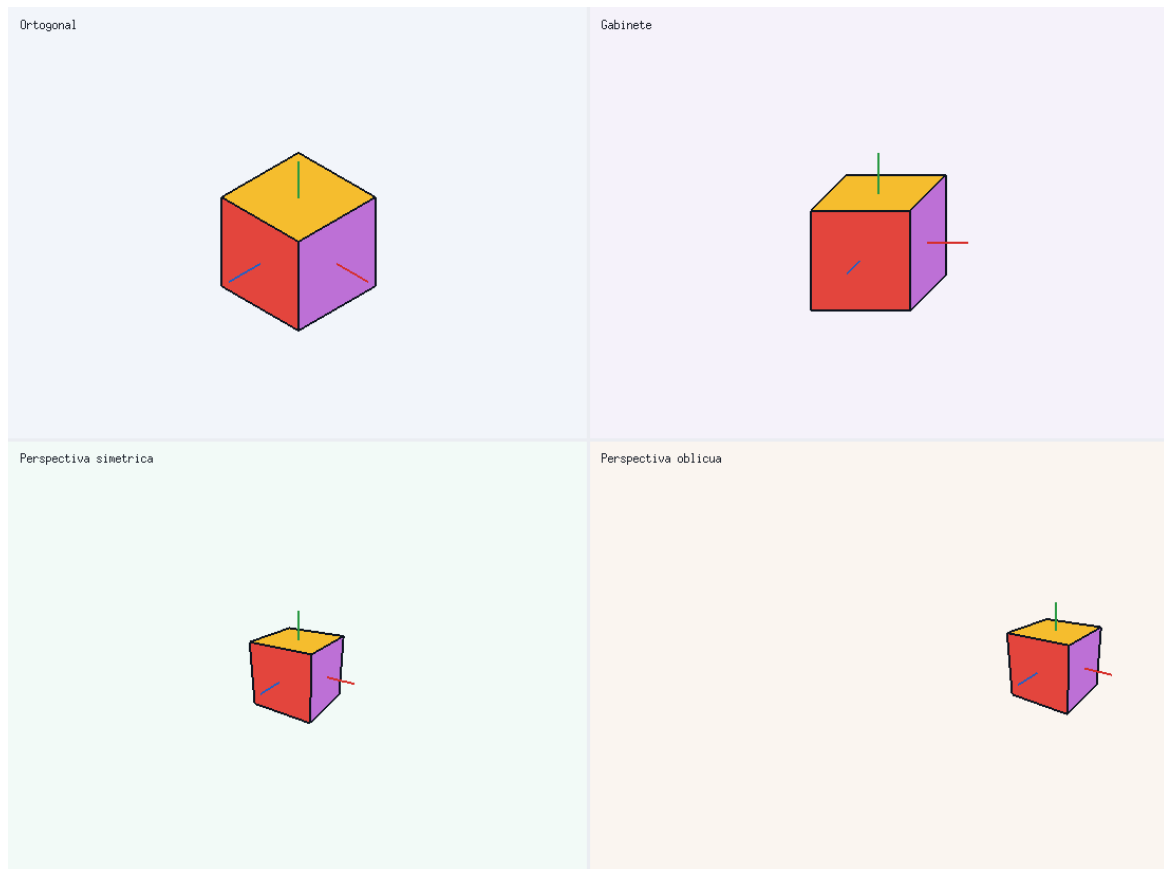


Figura 1: Ventana principal del programa con cuatro vistas del mismo cubo.

La Figura 1 muestra la ventana principal del programa. En el cuadrante superior izquierdo se observa la proyección ortogonal; en el superior derecho, la proyección gabinete; en el inferior izquierdo, la perspectiva simétrica; y en el inferior derecho, la perspectiva oblicua. Las etiquetas permiten identificar cada caso y los colores del cubo ayudan a comparar el comportamiento espacial de las caras en las cuatro vistas.

La vista ortogonal conserva el paralelismo de las aristas. La gabinete mantiene una cara frontal dominante y desplaza la profundidad en diagonal. La perspectiva simétrica muestra una proyección centrada, mientras que la perspectiva oblicua desplaza el volumen visible mediante un *frustum* asimétrico. Aunque las dos perspectivas conservan una pose cercana del cubo, su diferencia se encuentra en la configuración del volumen de proyección.

6. Visualización complementaria: hiperboloide de una hoja

Además de la ventana principal con las cuatro proyecciones del cubo, se documenta una visualización complementaria desarrollada dentro del laboratorio. Esta sección tiene como finalidad dejar evidencia de la aplicación de OpenGL en una geometría tridimensional más com-

pleja que un sólido básico, reutilizando la lógica de ejes, representación espacial y construcción gráfica de una escena 3D.

La superficie representada corresponde a un hiperboloide de una hoja definido por la ecuación:

$$\frac{x^2}{4} + \frac{(y-3)^2}{9} - \frac{(z+1)^2}{16} = 1$$

Esta expresión describe una superficie cuadrática centrada en $(0, 3, -1)$. Para su visualización se emplea una representación alámbrica, ya que la malla permite observar la continuidad de la superficie y su curvatura sin ocultar la estructura geométrica. En términos prácticos, este tipo de representación es útil para comprobar cómo una parametrización matemática puede transformarse en coordenadas renderizables dentro de una escena OpenGL.

Una forma conveniente de generar la superficie consiste en parametrizarla mediante funciones hiperbólicas:

$$x = 2 \cosh(v) \cos(u), \quad y = 3 + 3 \cosh(v) \sin(u), \quad z = -1 + 4 \sinh(v)$$

con u recorriendo el ángulo alrededor del eje de la superficie y v controlando la apertura de la malla. Al acotar el intervalo de v , se obtiene una porción visible y manejable de la superficie, evitando que la figura crezca de forma excesiva dentro de la ventana.

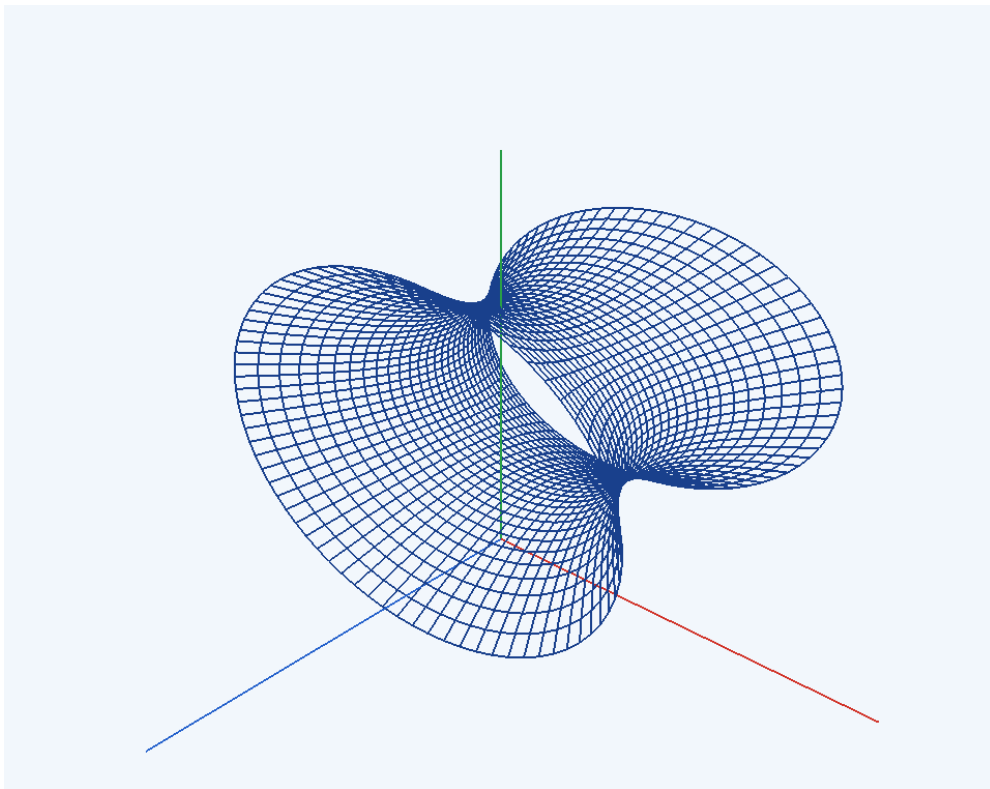


Figura 2: Representación alámbrica de un hiperboloide de una hoja en OpenGL.

La Figura 2 complementa la entrega al mostrar que el laboratorio no se limita al cubo em-

pleado para comparar proyecciones, sino que también permite representar superficies matemáticas mediante mallas paramétricas. Esta sección refuerza la relación entre geometría, cálculo vectorial y visualización computacional, que es uno de los aspectos prácticos de la informática gráfica.

7. Verificación de requisitos

Tabla 1: Relación entre el requisito de la actividad y la solución implementada.

Requisito	Cumplimiento
Ventana con cuatro vistas	La ventana se divide en cuatro cuadrantes mediante <code>glViewport()</code> y <code>glScissor()</code> .
Mismo cubo en todas las vistas	La escena se dibuja desde una función común que reutiliza el cubo, las aristas y los ejes.
Proyección ortogonal	Se usa una matriz ortográfica y una pose axonométrica del objeto.
Proyección gabinete	Se combina una matriz ortográfica con una cizalla de profundidad reducida.
Perspectiva simétrica	Se usa un <i>frustum</i> centrado con límites simétricos.
Perspectiva oblicua	Se usa un <i>frustum</i> asimétrico para obtener una perspectiva <i>off-axis</i> .
No mover el ojo	No se usa <code>gluLookAt()</code> ; el objeto se traslada hacia $-Z$ dentro de la escena.
Archivo principal único	La implementación se concentra en <code>main.c</code> .

8. Funcionamiento del programa

8.1. ¿Funciona bien el programa?

Sí. El programa cumple el objetivo de la actividad, porque abre una única ventana OpenGL y muestra cuatro vistas simultáneas del mismo cubo. Cada cuadrante tiene una etiqueta propia y aplica una configuración de proyección distinta. Además, la solución evita el uso de `gluLookAt()` y mantiene el ojo en la posición por defecto, usando transformaciones del objeto y matrices de proyección para construir la escena.

Desde el resultado visual, la vista ortogonal es la más directa de validar porque mantiene las aristas paralelas. La vista gabinete representa la profundidad mediante una cizalla oblicua con reducción. La perspectiva simétrica se apoya en un *frustum* centrado, y la perspectiva oblicua se diferencia mediante un *frustum* asimétrico. En conjunto, el programa permite comparar las cuatro formas de proyección solicitadas sobre una misma geometría.

8.2. ¿Qué fallos existen?

No se identifican fallos funcionales que impidan cumplir la actividad. Sin embargo, se reconocen algunas limitaciones propias de la implementación:

- El programa depende de un entorno compatible con X11/GLX y OpenGL clásico, por lo que su ejecución directa está orientada a sistemas que soporten ese entorno gráfico.
- Las etiquetas se dibujan con una fuente bitmap sencilla mediante `glXUseXFont`; es suficiente para la práctica, pero no busca una presentación tipográfica avanzada.
- Los parámetros de escala, traslación y amplitud del *frustum* fueron ajustados manualmente para priorizar que las cuatro vistas se observaran completas y centradas.
- La diferencia entre la perspectiva simétrica y la perspectiva oblicua es moderada para no recortar el cubo dentro del cuadrante. Puede exagerarse aumentando la asimetría del *frustum*, aunque eso reduciría la estabilidad visual de la captura.

Estas limitaciones no contradicen el enunciado. Más bien, delimitan el alcance de una solución académica enfocada en demostrar las cuatro proyecciones solicitadas de forma clara y ejecutable.

9. Conclusiones

El informe permite evidenciar que la solución desarrollada responde al alcance solicitado: una ventana OpenGL con cuatro vistas de un mismo cubo y con proyecciones diferenciadas. La explicación se centró en el documento, en la captura de ejecución y en las decisiones técnicas principales necesarias para justificar el resultado.

La implementación es defendible porque separa la escena común de las matrices de proyección. Esta separación evita interpretar las cuatro vistas como simples rotaciones del cubo y permite explicar por qué cada cuadrante corresponde a una proyección distinta. La restricción sobre la cámara también se cumple, ya que no se mueve el ojo desde $(0, 0, 0)$ ni se utiliza `gluLookAt()`.

En términos de entrega, el programa funciona correctamente para el objetivo planteado. Las limitaciones identificadas se relacionan con el entorno gráfico, la sencillez del rotulado y el ajuste manual de parámetros visuales, pero no afectan el cumplimiento central de la actividad.

La visualización del hiperboloide complementa el documento porque amplía la evidencia práctica del laboratorio hacia la representación de superficies paramétricas. Con ello se muestra que la implementación no solo permite comparar proyecciones sobre un cubo, sino también trasladar conceptos matemáticos a una escena 3D renderizada con OpenGL.

10. Archivos adjuntos

Junto con este informe se adjunta un archivo comprimido con el código fuente completo de la solución, incluyendo el archivo `main.c`. Además, se anexa el enlace al repositorio de GitHub que contiene el laboratorio:

<https://github.com/smaje99/computer-graphics-and-visualization-unir>